UNIVERSIDAD TECNOLÓGICA DE CHIHUAHUA

Entornos Virtuales y Negocios Digitales

Programación de Aplicaciones Web Progresivas

Investigación y configuración de las API y bases de datos del proyecto SIMA

PRESENTA:
Ángel Iván López Delgado
Celeste Guadalupe Ramos Gurrola
Ángel Esteban Cervantes Muñoz
PROFESOR(A): Juan Carlos Bustamante Lozano

Chihuahua, Chih., a 19 de octubre de 2026

Investigación y configuración de las API y bases de datos del proyecto SIMA

1. Introducción

SIMA (Sistema de Inteligencia de Movilidad y Aforo) es el proyecto integrador del Equipo 5. El sistema convierte las cámaras que un negocio ya tiene en sensores de afluencia: cuenta cuántas personas hay por zona, cuántas entran y salen y cuánto tiempo permanecen. Esa información la consume una aplicación web progresiva (PWA) que muestra el estado en vivo y el histórico.

Para que la PWA muestre datos, el proyecto se apoya en cuatro servicios: una API propia construida con FastAPI, un canal en tiempo real por WebSocket, una base de datos MongoDB alojada en Atlas y un conjunto de bibliotecas de visión computacional (Ultralytics YOLO y las herramientas de Roboflow) que producen los datos. Este reporte investiga cada uno, explica por qué se eligió y documenta los comandos y la configuración necesarios para conectarlo con el *framework* de la aplicación.

2. Objetivo

Investigar las API y bases de datos que utiliza SIMA y documentar la configuración necesaria para conectarlas con la PWA, para que cualquier integrante del equipo pueda reproducir el entorno y entender qué papel cumple cada pieza.

3. Arquitectura general

El sistema se divide en dos partes que se comunican por red. El *backend* recibe el video, detecta personas y publica eventos; la PWA los muestra.

Tabla 1. Flujo de datos de SIMA

Paso	Pieza	Qué hace
1	Cámara (archivo de video o RTSP)	Entrega los cuadros de video
2	Ultralytics YOLO	Detecta a cada persona en el cuadro
3	Roboflow `trackers` (ByteTrack)	Le asigna un identificador temporal a cada persona para seguirla entre cuadros

Paso	Pieza	Qué hace
4	Roboflow `supervision`	Calcula ocupación por zona y cruces de la línea de la puerta
5	API FastAPI	Publica los eventos por WebSocket y expone la configuración por REST
6	MongoDB Atlas	Guarda los eventos para consultar el histórico
7	PWA (Next.js)	Muestra el estado en vivo y las gráficas históricas

Fuente: elaboración propia con base en el repositorio del proyecto.

La regla de diseño es que **el *backend* escribe y la web solo lee**. La PWA nunca modifica datos; con eso se simplifican los permisos y se evita que un error en la interfaz corrompa el histórico.

4. Investigación de las API y bases de datos

4.1 API REST con FastAPI

Qué es. FastAPI es un *framework* de Python para construir API web. Valida automáticamente los datos de entrada y salida a partir de modelos de Pydantic y genera su propia documentación interactiva en la ruta `/docs` (FastAPI, s. f.).

Por qué se eligió. El procesamiento de video se escribe en Python, porque ahí están las bibliotecas de visión. Usar FastAPI permite que la API viva en el mismo proceso que el modelo, sin un servicio intermedio. Además soporta WebSocket de forma nativa, lo que cubre el tiempo real sin otra herramienta.

Qué expone SIMA.

Tabla 2. Rutas de la API de SIMA

Método	Ruta	Para qué la usa la PWA
GET	`/health`	Saber si el servidor está en línea
GET	`/api/layout`	Obtener las zonas y líneas de la cámara para dibujarlas sobre la imagen
WS	`/ws/events`	Recibir los eventos en tiempo real
GET	`/docs`	Documentación interactiva generada por FastAPI

4.2 WebSocket para el tiempo real

Qué es. WebSocket es un protocolo que mantiene una conexión abierta entre el navegador y el servidor, de modo que el servidor puede enviar datos sin que el cliente los pida cada vez (MDN Web Docs, s. f.).

Por qué se eligió. La ocupación cambia cada segundo. Consultar la API REST en un ciclo (*polling*) generaría cientos de peticiones vacías por minuto. Con WebSocket el servidor empuja el evento solo cuando ocurre.

Qué eventos envía.

Tabla 3. Eventos que publica SIMA

Tipo	Cuándo se emite	Campos
`occupancy`	Cada segundo	`total`, `zones[{zone_id, count}]`
`crossing`	Cuando alguien cruza la línea de una puerta	`line_id`, `direction` (`in`/`out`), `tracker_id`
`dwell`	Cuando alguien sale de una zona	`zone_id`, `tracker_id`, `seconds`

4.3 MongoDB Atlas como base de datos

Qué es. MongoDB es una base de datos orientada a documentos: guarda registros en formato parecido a JSON, sin un esquema rígido de tablas. Atlas es su servicio en la nube, que se encarga del servidor, los respaldos y la seguridad (MongoDB, s. f.-a).

Por qué se eligió. Los eventos de SIMA ya nacen como JSON, así que se guardan tal cual, sin conversión. MongoDB ofrece además **colecciones de series de tiempo** (*time-series*), optimizadas para datos que llegan ordenados por fecha y se consultan agrupados por hora o por día (MongoDB, s. f.-b), que es exactamente la forma de las gráficas del tablero. Se usó Atlas en lugar de una instalación local para que los tres integrantes trabajen sobre la misma base sin montar infraestructura.

Cómo se organizan los datos.

Tabla 4. Colecciones de la base `sima`

Colección	Tipo	Contenido
`crossings`	Serie de tiempo	Una entrada o salida por documento
`dwells`	Serie de tiempo	Una visita a una zona y cuántos segundos duró

Colección	Tipo	Contenido
`occupancy`	Serie de tiempo	Personas por zona, una muestra cada 5 segundos
`stores`	Normal	Cada negocio y sus cámaras
`cameras`	Normal	Zonas y líneas de cada cámara

Todos los documentos llevan `store_id` y `camera_id`, de modo que un negocio con varias cámaras se suma filtrando por negocio. Las horas se guardan en tiempo universal (UTC) y se convierten a la hora de Chihuahua (UTC−6) al mostrarlas.

Para no saturar la base, la ocupación se guarda una vez cada cinco segundos aunque el WebSocket la envíe cada segundo. Lo que pasa en este instante se lee del WebSocket; lo que ya pasó, de MongoDB.

4.4 Bibliotecas de visión: Ultralytics YOLO y Roboflow

No son API web, pero son la fuente de todos los datos, por eso se documentan.

Tabla 5. Bibliotecas de visión computacional

Biblioteca	Función en SIMA	Licencia
Ultralytics YOLO (modelo `yolo11n`)	Detectar personas en cada cuadro	AGPL-3.0
Roboflow `trackers` (ByteTrack)	Seguir a cada persona entre cuadros con un identificador	Apache 2.0
Roboflow `supervision`	Zonas poligonales (ocupación) y líneas (entradas y salidas)	MIT

Fuente: documentación oficial de cada proyecto (Roboflow, s. f.-a, s. f.-b; Ultralytics, s. f.).

La licencia de YOLO obliga a publicar el código si el sistema se ofrece como servicio. Por eso el detector está detrás de una interfaz intercambiable: el equipo puede migrar a RF-DETR (Apache 2.0) antes de un piloto comercial sin tocar el resto del sistema.

5. Configuración y conexión con el *framework*

5.1 Backend (Python + FastAPI)

El proyecto usa `uv` como gestor de dependencias de Python.

Tabla 6. Comandos de instalación del *backend*

#	Comando	Qué hace
1	`uv sync --extra yolo`	Instala FastAPI, Uvicorn, Pydantic, `supervision`, `trackers`, `pymongo` y YOLO
2	`cp .env.example .env`	Crea el archivo de configuración local
3	`uv run sima-run data/tienda.mp4`	Procesa un video y muestra los eventos en la terminal
4	`SIMA_RUN_PIPELINE=true uv run uvicorn sima.main:app --reload`	Levanta la API con el procesamiento de video activo

Las variables de conexión van en `.env`, que no se sube al repositorio:

Tabla 7. Variables de entorno de la conexión

Variable	Ejemplo	Para qué sirve
`SIMA_MONGO_URI`	`mongodb+srv://<usuario>:<clave>@<cluster>.mongodb.net`	Cadena de conexión a Atlas. Vacía = no se guarda nada
`SIMA_MONGO_DB`	`sima`	Nombre de la base
`SIMA_MONGO_OCCUPANCY_EVERY_S`	`5`	Cada cuántos segundos se guarda la ocupación
`SIMA_CORS_ORIGINS`	`["http://localhost:3000"]`	Dominios de la PWA autorizados a consultar la API

Al arrancar, el *backend* crea por sí solo las colecciones de series de tiempo si no existen y registra el negocio y la cámara. Si MongoDB deja de responder, el conteo

sigue funcionando y solo se pierden los eventos de ese lapso, que quedan registrados en el log.

5.2 Base de datos (MongoDB Atlas)

1. Crear una cuenta y un *cluster* gratuito (M0) en `cloud.mongodb.com`.
2. En **Database Access**, crear un usuario de base de datos con contraseña.
3. En **Network Access**, autorizar la IP del servidor (en desarrollo, la del equipo de cada integrante).
4. En **Connect** → **Drivers**, copiar la cadena de conexión `mongodb+srv://…` y pegarla en `SIMA_MONGO_URI`.
5. Verificar con `uv run sima-run data/tienda.mp4 --mongo` y revisar en **Browse Collections** que aparezcan las colecciones.

5.3 PWA (Next.js)

La PWA se conecta por dos caminos distintos según el tipo de dato.

Tabla 8. Cómo obtiene cada dato la PWA

Dato	Origen	Cómo se conecta
Estado en vivo	WebSocket `/ws/events`	Desde el navegador, con la API nativa `WebSocket`
Zonas de la cámara	REST `/api/layout`	Desde el navegador, con `fetch`
Histórico y gráficas	MongoDB Atlas	Desde una ruta del servidor de Next.js, nunca desde el navegador

Tabla 9. Comandos y configuración del lado de la PWA

#	Comando / archivo	Qué hace
1	`yarn add mongodb`	Instala el *driver* oficial de MongoDB para Node.js
2	`.env.local` con `MONGODB_URI=…`	Guarda la cadena de conexión solo en el servidor
3	`.env.local` con `NEXT_PUBLIC_SIMA_API=http://localhost:8000`	Dirección pública de la API de FastAPI

#	Comando / archivo	Qué hace
4	`app/api/<ruta>/route.ts`	Rutas del servidor que consultan MongoDB y devuelven JSON a la interfaz

La cadena de conexión de MongoDB **no lleva el prefijo `NEXT_PUBLIC_`**: en Next.js, solo las variables con ese prefijo llegan al navegador (Vercel, s. f.). Así la contraseña de la base nunca queda expuesta en el código que descarga el usuario.

5.4 Caché de la API en la PWA

El *service worker* de la PWA aplica una regla distinta a cada fuente:

Tabla 10. Estrategia de caché por fuente

Fuente	Estrategia	Razón
`/api/layout`	Primero la red, copia guardada si no hay conexión	Cambia poco; sin red se pueden seguir dibujando las zonas
Histórico (rutas que leen MongoDB)	Primero la red, copia guardada si no hay conexión	Se prefiere el dato fresco, pero no una pantalla vacía
WebSocket `/ws/events`	Sin caché	Un dato en vivo guardado es un dato falso; sin red se muestra "sin conexión"

6. Problemas identificados y su solución

Tabla 11. Problemas de conexión y solución aplicada

#	Problema	Causa	Solución
1	El navegador bloqueaba las peticiones de la PWA a la API	La API no autorizaba el origen `localhost:3000` (CORS)	Se configuró `CORSMiddleware` en FastAPI con la lista `SIMA_CORS_ORIGINS`
2	Las gráficas por hora salían corridas seis horas	MongoDB guarda en UTC	Las consultas agrupan con `timezone: "America/Chihuahua"`
3	El mismo `tracker_id` aparecía en días	El identificador del *tracker* solo es único	Los conteos de personas distintas se hacen por corrida, no por

#	Problema	Causa	Solución
	distintos como si fuera la misma persona	dentro de una corrida	identificador entre días
4	Riesgo de exponer la contraseña de la base	Variable de entorno pública por error	La cadena de Atlas solo existe en `.env` del *backend* y en rutas del servidor de Next.js

7. Conclusiones

SIMA separa con claridad dos tipos de dato y eso determina qué tecnología se usa para cada uno. Lo que pasa en este segundo viaja por WebSocket y no se guarda en caché; lo que ya pasó se guarda en MongoDB y se consulta desde el servidor. Mezclar ambos, por ejemplo consultando la base cada segundo para simular el tiempo real, habría multiplicado el costo y la latencia sin ningún beneficio.

La elección de MongoDB Atlas respondió a la forma de los datos: los eventos nacen como JSON y se consultan agrupados por tiempo, que es el caso para el que existen las colecciones de series de tiempo. La parte más delicada de la configuración no fue instalar las dependencias, sino decidir dónde vive cada credencial y qué dominios pueden consultar la API. Esas dos decisiones son las que hacen que el sistema sea seguro cuando pase del laboratorio a un negocio real.

Referencias

FastAPI. (s. f.). *FastAPI documentation*. Recuperado el 19 de octubre de 2026, de https://fastapi.tiangolo.com/

MDN Web Docs. (s. f.). *The WebSocket API (WebSockets)*. Mozilla. Recuperado el 19 de octubre de 2026, de https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

MongoDB. (s. f.-a). *MongoDB Atlas documentation*. Recuperado el 19 de octubre de 2026, de https://www.mongodb.com/docs/atlas/

MongoDB. (s. f.-b). *Time series collections*. Recuperado el 19 de octubre de 2026, de https://www.mongodb.com/docs/manual/core/timeseries-collections/

Roboflow. (s. f.-a). *Supervision* [Software]. GitHub. Recuperado el 19 de octubre de 2026, de https://github.com/roboflow/supervision

Roboflow. (s. f.-b). *Trackers* [Software]. GitHub. Recuperado el 19 de octubre de 2026, de https://github.com/roboflow/trackers

Ultralytics. (s. f.). *Ultralytics YOLO docs*. Recuperado el 19 de octubre de 2026, de https://docs.ultralytics.com/

Vercel. (s. f.). *Environment variables*. Next.js documentation. Recuperado el 19 de octubre de 2026, de https://nextjs.org/docs/app/guides/environment-variables